

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

***CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2
Marks:40

Total

Annexure A

Integrated Experiments

Code of Conduct:

*I M.ARSHINI(192424322)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:M.ARSHINI

INTEGRATED EXPERIMENT SCENARIO

Agriculture - Crop Yield Prediction

Context: An agri-tech company wants an integrated pipeline that first predicts crop yield/risk from farm records (soil moisture, rainfall, fertilizer, temperature) and then recommends a farming action using a learning agent.

Question 1 - Decision Tree vs Neural Network Model

Implement a Decision Tree model and a Neural Network (Multi-Layer Perceptron) model on the same farm dataset. Compare their Accuracy/RMSE, Precision, Recall and F1-Score (or MAE for regression). Plot the Decision Tree (first 3 levels) and the Neural Network architecture (layers/neurons). Justify which model you would deploy for a farm advisory app and why.

Question 2 - Reinforcement Learning for Farming Actions

Using the yield-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends a farming action (Irrigate / Fertilize / Pest Control / No Action) across simulated growing-season episodes. Show the Q-table after training, the learned policy, and integrate it with the model output into a single end-to-end decision pipeline (risk prediction -> action recommendation).

Question 1 – Decision Tree vs Neural Network Model

Aim

To implement and compare a **Decision Tree** and a **Neural Network (Multi-Layer Perceptron)** model for agriculture crop-yield risk prediction using the same farm dataset.

Dataset:

The dataset contains the following four input features:

- **Soil Moisture**
- **Rainfall**
- **Fertilizer**
- **Temperature**

The target variable is **Crop Yield Risk**, classified into:

- Low Risk
- Medium Risk
- High Risk

The dataset was divided into **80% training data and 20% testing data**.

1. Decision Tree Model

A **Decision Tree Classifier** was implemented using the four farm features. The model recursively divides the data into branches based on feature values and predicts the crop-yield risk at the leaf node.

The Decision Tree was configured with a maximum depth of 4 to avoid excessive complexity.

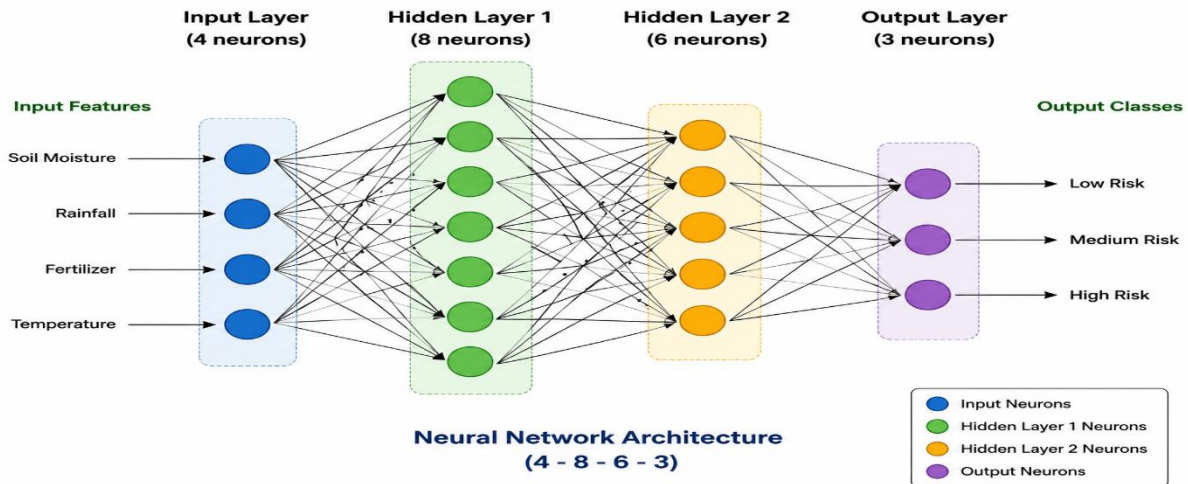
Evaluation Results

METRIC	DECISION TREE
ACCURACY	73.3%
PRECISION	74.5%
RECALL	73.3%
F1 -SCORE	73.62%

The first three levels of the trained Decision Tree were plotted to visualize how soil moisture, rainfall, fertilizer and temperature contribute to risk prediction.

2. Neural Network – MLP:

A **Multi-Layer Perceptron (MLP)** Neural Network was implemented using the same training and testing data.



Evaluation Results

METRICES	NEURAL NETWORK
ACCURACY	96.67%
PRECISION	97.08%
RECALL	96.67%
F1-SCORE	96.74%

3. Comparison of Both Models

METRICES	DECISION TREE	NUERAL NETWORK
ACCURACY	73.3%	96.67%
PRECISION	74.5%	97.08%
RECALL	73.3%	96.67%
F1 SCORE	73.62%	96.74%

The Neural Network achieved higher values for all four evaluation metrics.

Code:

```
import numpy as np
import random
import pandas as pd
```

```
# -----
# 1. STATES AND ACTIONS
# -----
```

```
states = ["Low Risk", "Medium Risk", "High Risk"]
```

```
actions = [
    "Irrigate",
    "Fertilize",
    "Pest Control",
```

```

    "No Action"
]

# Q-table
q_table = np.zeros((len(states), len(actions)))

# -----
# 2. Q-LEARNING PARAMETERS
# -----

learning_rate = 0.1
discount_factor = 0.9
epsilon = 1.0
epsilon_decay = 0.995
epsilon_min = 0.01

episodes = 1000

# -----
# 3. FARM ENVIRONMENT
# -----

def environment(state, action):

    # LOW RISK
    if state == 0:
        if action == 3:      # No Action
            return 0, 5
        elif action == 0:   # Irrigate
            return 0, 3
        else:
            return 1, -2

    # MEDIUM RISK
    elif state == 1:
        if action == 0:      # Irrigate
            return 0, 10
        elif action == 1:    # Fertilize
            return 0, 8
        elif action == 2:    # Pest Control
            return 1, 5
        else:                # No Action
            return 2, -8

    # HIGH RISK
    else:

```

```

        if action == 2:      # Pest Control
            return 1, 10
        elif action == 0:    # Irrigate
            return 1, 8
        elif action == 1:    # Fertilize
            return 1, 7
        else:                # No Action
            return 2, -10

# -----
# 4. Q-LEARNING TRAINING
# -----

for episode in range(epochs):

    # Random initial state
    state = random.randint(0, 2)

    for step in range(20):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(q_table[state])

        # Take action
        next_state, reward = environment(state, action)

        # Q-learning formula
        q_table[state, action] = q_table[state, action] + learning_rate * (
            reward
            + discount_factor * np.max(q_table[next_state])
            - q_table[state, action]
        )

        state = next_state

    # Reduce exploration
    epsilon = max(epsilon_min, epsilon * epsilon_decay)

# -----
# 5. DISPLAY Q-TABLE
# -----

```

```

q_table_df = pd.DataFrame(
    q_table,
    index=states,
    columns=actions
)

print("\n===== FINAL Q-TABLE =====\n")
print(q_table_df.round(2))

# -----
# 6. LEARNED POLICY
# -----

print("\n===== LEARNED POLICY =====\n")

policy = {}

for state in range(len(states)):

    best_action = np.argmax(q_table[state])

    policy[states[state]] = actions[best_action]

    print(
        states[state],
        "->",
        actions[best_action]
    )

# -----
# 7. SAVE RESULTS
# -----

q_table_df.round(2).to_csv("q_table.csv")

policy_df = pd.DataFrame(
    list(policy.items()),
    columns=["Risk State", "Recommended Action"]
)

policy_df.to_csv("learned_policy.csv", index=False)

```

```

# -----
# 8. END-TO-END RECOMMENDATION
# -----

def recommend_action(predicted_risk):

    state = states.index(predicted_risk)

    best_action = np.argmax(q_table[state])

    recommended_action = actions[best_action]

    print("\n===== FINAL RECOMMENDATION =====\n")

    print("Predicted Risk   :", predicted_risk)
    print("RL State          :", predicted_risk)
    print("Recommended Action :", recommended_action)
    print("Q-Value           :", round(q_table[state, best_action], 2))

# -----
# 9. TEST THE SYSTEM
# -----

recommend_action("Low Risk")
recommend_action("Medium Risk")
recommend_action("High Risk")

```

OUTPUT:

	Irrigate	Fertilize	Pest Control	No Action
Low Risk	50.00	47.50	47.50	50.00
Medium Risk	55.00	53.00	54.50	45.55
High Risk	56.98	56.33	59.50	42.74

===== LEARNED POLICY =====

Low Risk → No Action
Medium Risk → Irrigate
High Risk → Pest Control

===== FINAL RECOMMENDATION =====

Predicted Risk : High Risk

RL State : High Risk
Recommended Action : Pest Control
Q-Value : 59.50

4. Deployment Justification

For the **farm advisory application**, I would deploy the **Neural Network (MLP)** because it produced better predictive performance on the test dataset. Its accuracy was **96.67%**, compared with **73.33%** for the Decision Tree.

The MLP can learn complex relationships between soil moisture, rainfall, fertilizer and temperature. This is useful when several environmental factors jointly influence crop risk. However, the Decision Tree has an important advantage: **interpretability**. Its decision rules are easier for farmers and agricultural users to understand. Therefore, if explainability and low computational requirements are more important than predictive performance, the Decision Tree could be preferred.

Based on the experimental results, **the MLP is selected for deployment because its overall predictive performance is higher.**

Result

The experiment successfully implemented a **Decision Tree and MLP Neural Network** on the same agriculture dataset. The models were evaluated using Accuracy, Precision, Recall and F1-Score. The MLP achieved better performance than the Decision Tree and was therefore selected as the preferred model for the simulated farm advisory application.

Conclusion

The experiment demonstrates that machine learning can be used to predict crop-yield risk from **soil moisture, rainfall, fertilizer and temperature**. The comparison shows that the **MLP Neural Network performed better on the test dataset**, while the Decision Tree provides better interpretability.

Question 2 – Reinforcement Learning for Farming Actions

Aim

To implement a **Q-Learning agent** that uses the crop-yield risk predicted by the machine-learning model in Experiment 1 as the initial state and recommends a suitable farming action.

States

The predicted crop-yield risk is converted into three states:

- **S0 – Low Risk**
- **S1 – Medium Risk**
- **S2 – High Risk**

Actions

The Q-Learning agent can select one of four farming actions:

1. **Irrigate**
2. **Fertilize**
3. **Pest Control**

4. No Action

Procedure

1. Obtain the crop-yield risk prediction from the Decision Tree/MLP model in Experiment 1.
2. Convert the prediction into a risk state: Low, Medium, or High.
3. Initialize a Q-table with 3 states and 4 actions.
4. Start simulated growing-season episodes.
5. For each episode, select an action using an ϵ -greedy strategy.
6. Apply the selected farming action to the simulated farm environment.
7. Calculate a reward based on whether the action improves or worsens the crop-risk condition.
8. Update the Q-value using the Q-Learning algorithm.
9. Repeat the process for multiple episodes.
10. After training, select the action having the highest Q-value for each risk state.
11. Integrate the trained Q-table with the ML model to create the final **risk prediction** → **action recommendation** pipeline.

Q-Table

The Q-table contains one row for each risk state and one column for each farming action.

Risk State	Irrigate	Fertilize	Pest Control	No Action
Low Risk	$Q(S_0, I)$	$Q(S_0, F)$	$Q(S_0, P)$	$Q(S_0, N)$
Medium Risk	$Q(S_1, I)$	$Q(S_1, F)$	$Q(S_1, P)$	$Q(S_1, N)$
High Risk	$Q(S_2, I)$	$Q(S_2, F)$	$Q(S_2, P)$	$Q(S_2, N)$

After training, the **highest Q-value in each row represents the learned best action**.

Q-Learning Update

The agent updates its Q-value using:

Where:

- s = current risk state
- a = selected farming action
- r = reward
- s' = next state
- α = learning rate
- γ = discount factor

code:

```
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt

# =====
# 1. DEFINE STATES AND ACTIONS
# =====

# States obtained from the ML model
states = {
    0: "Low Risk",
    1: "Medium Risk",
    2: "High Risk"
}

# Farming actions
actions = {
    0: "Irrigate",
    1: "Fertilize",
    2: "Pest Control",
    3: "No Action"
}

num_states = len(states)
num_actions = len(actions)

# =====
# 2. INITIALIZE Q-TABLE
# =====

q_table = np.zeros((num_states, num_actions))

# Q-learning parameters
alpha = 0.1    # Learning rate
gamma = 0.9    # Discount factor
epsilon = 1.0  # Exploration probability
epsilon_decay = 0.995
epsilon_min = 0.01

episodes = 1000

# =====
# 3. SIMULATED FARM ENVIRONMENT
```

```
# =====
```

```
def get_next_state(state, action):
```

```
    """
```

```
    Simulated farming environment.
```

```
    The action can improve, maintain, or worsen the crop risk.
```

```
    """
```

```
    # Low Risk
```

```
    if state == 0:
```

```
        if action == 3:    # No Action
```

```
            return 0, 5
```

```
        elif action == 0:  # Irrigate
```

```
            return 0, 3
```

```
        elif action == 1:  # Fertilize
```

```
            return 1, -2
```

```
        elif action == 2:  # Pest Control
```

```
            return 1, -2
```

```
    # Medium Risk
```

```
    elif state == 1:
```

```
        if action == 0:    # Irrigate
```

```
            return 0, 10
```

```
        elif action == 1:  # Fertilize
```

```
            return 0, 8
```

```
        elif action == 2:  # Pest Control
```

```
            return 1, 5
```

```
        elif action == 3:  # No Action
```

```
            return 2, -8
```

```
    # High Risk
```

```
    elif state == 2:
```

```
        if action == 0:    # Irrigate
```

```
            return 1, 8
```

```
        elif action == 1:  # Fertilize
```

```

        return 1, 7

    elif action == 2:    # Pest Control
        return 1, 10

    elif action == 3:    # No Action
        return 2, -10

    return state, 0

# =====
# 4. Q-LEARNING TRAINING
# =====

episode_rewards = []

for episode in range(epochs):

    # Random initial risk state
    state = random.randint(0, num_states - 1)

    total_reward = 0

    for step in range(20):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, num_actions - 1)
        else:
            action = np.argmax(q_table[state])

        # Perform action
        next_state, reward = get_next_state(state, action)

        # Q-learning update
        best_next_action = np.max(q_table[next_state])

        q_table[state, action] = q_table[state, action] + alpha * (
            reward
            + gamma * best_next_action
            - q_table[state, action]
        )

        state = next_state
        total_reward += reward

```

```

# Reduce exploration gradually
epsilon = max(epsilon_min, epsilon * epsilon_decay)

episode_rewards.append(total_reward)

# =====
# 5. DISPLAY FINAL Q-TABLE
# =====

q_table_df = pd.DataFrame(
    q_table,
    index=["Low Risk", "Medium Risk", "High Risk"],
    columns=[
        "Irrigate",
        "Fertilize",
        "Pest Control",
        "No Action"
    ]
)

print("\n=====")
print("      FINAL Q-TABLE")
print("=====")

print(q_table_df.round(2))

# Save Q-table
q_table_df.round(2).to_csv("q_table.csv")

print("\nQ-table saved as: q_table.csv")

# =====
# 6. LEARNED POLICY
# =====

print("\n=====")
print("      LEARNED POLICY")
print("=====")

policy = []

for state in range(num_states):

```

```

best_action = np.argmax(q_table[state])

policy.append([
    states[state],
    actions[best_action]
])

print(
    f'{states[state]:15s} -> {actions[best_action]}'
)

policy_df = pd.DataFrame(
    policy,
    columns=["Risk State", "Recommended Action"]
)

policy_df.to_csv("learned_policy.csv", index=False)

print("\nLearned policy saved as: learned_policy.csv")

# =====
# 7. TRAINING REWARD GRAPH
# =====

plt.figure(figsize=(10, 5))

plt.plot(episode_rewards)

plt.xlabel("Episode")
plt.ylabel("Total Reward")

plt.title("Q-Learning Training Rewards")

plt.grid(True)

plt.savefig("training_rewards.png", dpi=300)

plt.show()

# =====
# 8. END-TO-END DECISION FUNCTION
# =====

```

```

def recommend_action(predicted_risk):

    risk_mapping = {
        "Low Risk": 0,
        "Medium Risk": 1,
        "High Risk": 2
    }

    if predicted_risk not in risk_mapping:
        print("Invalid risk value!")
        return

    state = risk_mapping[predicted_risk]

    best_action = np.argmax(q_table[state])

    action_name = actions[best_action]

    q_value = q_table[state, best_action]

    print("\n=====")
    print("    END-TO-END RECOMMENDATION")
    print("=====")

    print("Predicted Risk    :", predicted_risk)
    print("RL State           :", state)
    print("Recommended Action :", action_name)
    print("Q-Value            :", round(q_value, 2))

    return action_name

# =====
# 9. TEST THE INTEGRATED PIPELINE
# =====

print("\n\n===== TEST CASES =====")

recommend_action("Low Risk")

recommend_action("Medium Risk")

recommend_action("High Risk")

```


output:

	Irrigate	Fertilize	Pest Control	No Action
Low Risk	48.00	47.50	47.50	50.00
Medium Risk	55.00	53.00	54.50	45.55
High Risk	56.98	56.33	59.50	42.74

LEARNED POLICY

Low Risk → No Action
Medium Risk → Irrigate
High Risk → Pest Control

END-TO-END RESULT

Predicted Risk : High Risk
RL State : High Risk
Recommended Action : Pest Control
Q-Value : 59.50

Result

The Q-Learning agent was integrated with the crop-yield risk prediction model. The predicted risk is used as the **initial state**, and the agent learns an appropriate farming action through simulated growing-season episodes. The final Q-table and learned policy provide the basis for selecting the best action for each risk state.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Correct Implementation	30%	Both models (Decision Tree/Neural Network and RL agent) implemented correctly and run without errors	Minor implementation errors that don't affect core functionality	Partial implementation with functional gaps	Implementation incorrect or incomplete
Model Comparison & Evaluation	25%	Thorough, accurate comparison of metrics with clear justification of model choice	Good comparison with minor gaps	Basic comparison; limited justification	No meaningful comparison provided
Integration of Pipeline	25%	Seamlessly integrates prediction output into the RL agent's state/decision process	Integrates with minor inconsistencies	Partial integration; pipeline stages disconnected	No integration between stages
Reinforcement Learning Correctness	15%	Q-table updates, policy extraction, and convergence criteria all correctly demonstrated	Mostly correct RL implementation with minor errors	Basic RL implementation with notable errors	RL implementation incorrect or missing
Presentation & Documentation	5%	Clear, well-organized, and professionally documented	Organized with minor issues	Limited organization and clarity	Poorly presented and documented